

CODEBASE WALKTHROUGH · SIDEQUEST · REACT + FIREBASE

Sidequest Under the Hood

Every moving part of the app you built, in the order it runs — and the reasoning behind each decision, phrased the way you'd say it out loud.

React 18

TypeScript

Vite 6

Firebase Auth

Cloud Firestore

Firestore Security Rules

Firebase Hosting

vite-plugin-pwa

Tailwind v4

Geoapify

lucide-react

01

THE PITCH

What the app is, and the one constraint that shapes all of it

Sidequest is a mobile web app for making spontaneous plans with friends. You post something small — coffee in thirty minutes, gym at five — address it to a group of friends, and it lands on their Explore tab where they can tap Join. If you marked the plan flexible, they can propose a different time or place instead, and you accept or decline.

The single most important architectural fact: **there is no backend server.** No Node service, no Cloud Functions, no API you wrote. The React app talks straight to Firestore, and the only thing standing between a user and the database is a file of security rules.

That was a deliberate constraint — Cloud Functions require a billing account, and this project stays on free tiers. Almost every interesting decision in the codebase is downstream of it. When you can't run code on a server, you can't fan out notifications, you can't validate a write in a

handler, and you can't join two users' private data together. Sections 5, 7, and 9 are all about what you did instead.

SAY THIS

"It's a serverless client app. Everything that would normally be a backend endpoint is either a client-side write guarded by a Firestore rule, or something I derive on the client from data I'm already subscribed to."

Where it came from

The project started as a Figma Make export — that's why `package.json` is still named `@figma/my-make-file`, why there's a `figmaAssetResolver` plugin in the Vite config, and why `components/ui/` holds ~45 shadcn/Radix components. That folder is a stock library that came with the export; the app's own screens barely use it. Everything in `data/`, `screens/` and `firestore.rules` is hand-written, and that's the part worth talking about.

02

STRUCTURE

How the app is put together

There are three layers, and the folder names match them exactly:

FOLDER	JOB
<code>src/app/data/</code>	Every Firestore read and write, plus the React contexts that hold the results. No JSX except providers.
<code>src/app/screens/</code>	Full-screen views: auth flow, and the five bottom-nav tabs.
<code>src/app/components/</code>	Cards, bottom sheets, small shared bits.
<code>src/app/types/index.ts</code>	The contract between the two. One file, every shared type.

One rule the codebase holds to: **screens never import**

firebase/firestore. They call a hook or a function from `data/`. That's why you can read `plans.ts` and understand the whole plan feature without opening a single component.

No router — the shell is a state machine

`react-router` is in `package.json` but never imported. Navigation is two pieces of state in `App.tsx`:

- `screen` — where you are in the funnel: `loading` → `welcome` → `signup` → `createProfile` → `onboarding` → `app`
- `tab` — which of the five tabs is showing, once `screen` `===` `"app"`

Overlays (notification drawer, plan detail, edit sheet, suggest sheet, cancel dialog) are held as nullable state in the same component and rendered conditionally on top. They live at the shell rather than inside the cards that open them, because a dialog opened from inside Home's scroll container would get clipped by it.



Fig. 1 — Why the providers nest in this order. Every query in the app filters on your `uid`, so nothing inside can subscribe until `onAuthStateChanged` has fired. Nesting encodes that dependency in the component tree instead of leaving it to a guard clause in each hook.

03

THE
DATABASE

What actually lives in Firestore

Four top-level collections. That's the whole database.

PATH	HOLDS
<code>users/{uid}</code>	name, username, email, radiusMiles, status
<code>users/{uid}/friends/{friendUid}</code>	one doc per friend – private to you
<code>users/{uid}/groups/{groupId}</code>	named bundles of friend uids – private to you
<code>usernames/{handle}</code>	{ uid } – a handle reservation
<code>friendRequests/{from}_{to}</code>	a pending invitation; existing <i>is</i> the pending state
<code>plans/{planId}</code>	the plan, its audience, its roster, and its suggestions

The thing worth pointing out in an interview is the **shape of a plan document**. A plan doesn't reference an attendees collection – it carries four parallel maps and arrays inside itself:

- `audienceUids: string[]` – who was invited
- `attendeeUids: string[]` – who's in
- `attendeeNames: { uid: name }` and `joinedAt: { uid: Timestamp }`
- `suggestions: { uid: {...} }` – proposed changes, keyed by who made them

WHY DENORMALIZED

Firestore charges per document read and has no joins. A subcollection of attendees would mean an extra read per plan per render, plus a second listener. Flattening them onto the plan means **one document read gives you the whole plan, including who's coming** – and it makes the security rules possible, because a rule

can compare the before and after of a field but can't inspect another document's write.

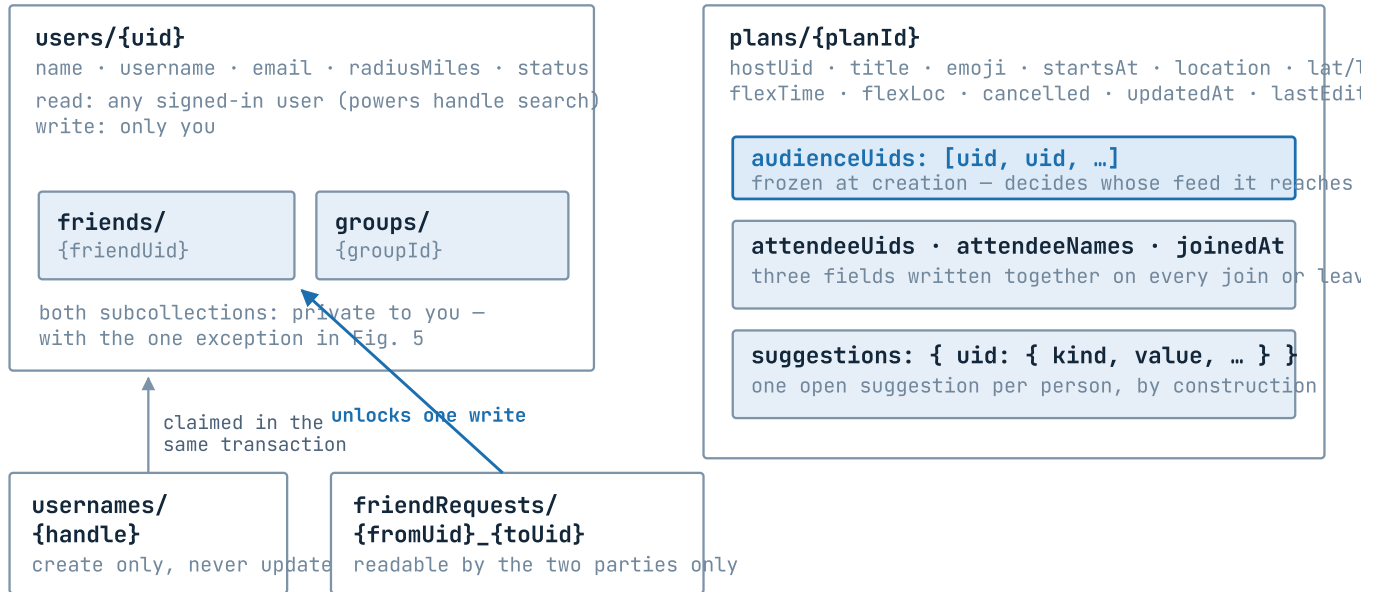


Fig. 2 – The whole schema. Note what is *not* here: no notifications collection, no attendees collection, no memberships table. Each of those was replaced by a field on a document you were already reading.

04

IDENTITY

Signing up, and staying signed in

Signup is deliberately two steps, because Firebase Auth and your profile data are two different systems.

Step one — the account. SignupScreen calls `signUpWithEmail()`, or one of the OAuth popups for Google and Apple. At this point Firebase Auth has a uid for you, but Firestore has nothing.

Step two — the profile. CreateProfileScreen collects a name and a handle and calls `saveProfile()`, which is the one place in the codebase that uses a **transaction**:

- `read usernames/{handle}`
- if it exists and points at someone else, throw `handle-taken`
- otherwise write both `usernames/{handle}` and `users/{uid}` together

WHY A TRANSACTION

Two people typing the same handle at the same moment would both read "free" and both write. A transaction makes Firestore re-run the whole block if the document it read changed underneath it, so exactly one of them wins. It's a classic check-then-act race, and a transaction is the standard fix — good thing to be able to name.

The boot problem, and the splash screen

Restoring a session is asynchronous. On first paint, Firebase genuinely does not yet know whether you're signed in. If you route on that, an already-signed-in user sees the welcome screen flash past before being bounced into the app.

So `AuthStatus` has three values, not two: `"loading" | "signedOut" | "signedIn"`. The shell holds a splash for `loading`, and routes once, guarded by a `useRef`:

- `signedOut` → welcome
- `signedIn` and a saved `name` exists → straight into the app
- `signedIn` but no `name` → back to profile setup

That third branch is the one people forget: someone who created an account and closed the tab before picking a handle. `hasProfile` is `Boolean(profile.name)` — "has an account" and "finished signing up" are different questions, and the app asks the second one.

The `booted` ref matters too. Without it, the effect would re-fire on every auth change and yank you back to a screen you'd navigated away from — including on sign-out, where `ProfileTab` sets the screen itself.

Onboarding

Two steps: request the browser geolocation permission (advancing either way — a declined prompt shouldn't trap anyone), then search handles and send friend requests to whoever you pick. A third step for alert radius exists in the code but is commented out, waiting on a feature that isn't built (§12).

05

FRIENDS

A mutual handshake with nobody in the middle

This is the best "I had a real constraint and solved it" story in the project, so it's worth being fluent in.

Friendship is mutual: if Bob accepts Alice, Alice must appear on Bob's list *and* Bob on Alice's. But `users/{uid}/friends/` is private — normally only you can write to your own. With no server to perform the second write, how does Alice's list ever get updated?

The request document

Its ID is derived, not random: `friendRequests/{fromUid}_{toUid}`. Two consequences fall straight out of that:

- Asking twice overwrites the same document instead of stacking duplicates.
- A rule can check `requestId = request.auth.uid + '_' + toUid`, which makes it impossible to forge a request that looks like it came from someone else.

There's also **no status field**. The document existing *is* the pending state; accepting and ignoring both delete it. So there's no way to end up with a row that says "pending" long after it was answered.

The rule exception

In `firestore.rules`, the `friends` subcollection carries one narrow

create allowance:

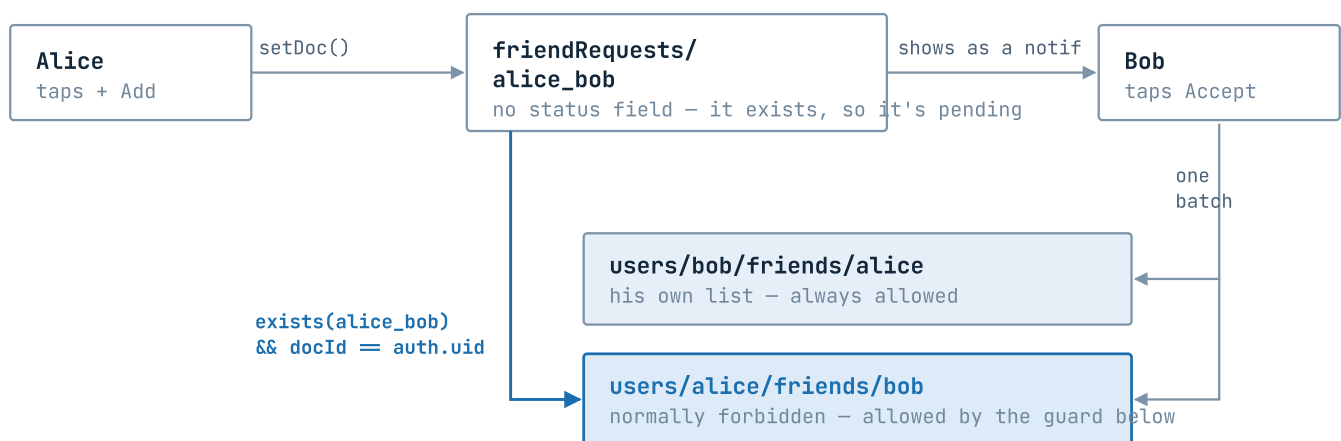
- the new document's ID must equal the caller's own uid — so you can only ever add *yourself* to someone's list
- and `friendRequests/{owner}_{caller}` must exist — so you can only do it while their invitation to you is open

`acceptFriendRequest()` then does both writes in a `writeBatch`, so there's no window where the friendship is half-formed. The request is deleted *after* the batch commits, not inside it — whether a rule's `exists()` can still see a document the same batch is deleting is a subtlety the author didn't want to bet the feature on.

SAY THIS

"I couldn't do a server-side fan-out, so I let the recipient perform the second write themselves, and used the pending request document as the capability that authorizes it. The rule grants exactly one write, to exactly one path, for exactly as long as the invitation is open."

One more edge case handled in code rather than rules: if Alice and Bob request each other simultaneously, whoever accepts first can't delete the other direction's request. So `FriendRequestProvider` filters out incoming requests from people already on your friend list.



then, after the batch commits: delete `friendRequests/alice_bob`

Fig. 5 – The pending request is a capability. The highlighted write is the one that would normally be impossible. It is authorized not by who Bob is, but by the existence of a document that Alice created and that will be deleted a moment later.

06

PLANS

Creating, addressing, and joining a plan

Creating

CreateTab is a progressive four-step form — name, location, time, audience — where each step reveals itself as the one before is answered. Typing a title alone advances you to step two, so the fast path really is a few taps.

On Share, `createPlan()` resolves coordinates (from the picked place, or from your own position for a plan set "here"), stamps the audience, and writes one document. Two details are worth knowing:

- **Undefined is spread in conditionally.** Firestore rejects an undefined value outright, so `lat / lng` use `...(typeof lat === "number" ? { lat, lng } : {})` rather than being set to undefined.
- **You go on your own roster immediately**, as attendee one. That's what lets Home read hosting and joining through the same code path.

Addressing — the key idea

The plan carries `audienceUids`: a frozen list of the friend uids it was sent to, resolved at write time from either your whole friend list or one group's members.

WHY FREEZE THE AUDIENCE

Firestore can't join a plan against someone else's private friend list at read time — the reader isn't allowed to see it. So the audience has to be materialized onto the plan when it's created. The bonus: a

friend you add tomorrow doesn't retroactively appear on a plan that already happened.

Reading it back — two queries, not one

`watchMyPlans()` runs **two** live subscriptions and merges them in a `Map` keyed by document ID:

- `where("hostUid", "=", uid)` — plans you host
- `where("attendeeUids", "array-contains", uid)` — plans you joined

Each snapshot owns its own map, so a document leaving one query can't delete the copy the other still holds. Nothing renders until *both* have reported once — otherwise Home paints your hosted plans and then visibly grows a moment later.

Sorting happens in JavaScript, not in the query. Pairing a `where` with an `orderBy` on a different field requires a composite index you'd have to go and create in the console; at this list size it isn't worth it. Same reasoning appears in `friends.ts` for a different reason — a Firestore `orderBy` silently drops documents missing that field, so a friend saved before `name` existed would just vanish.

`watchFriendPlans()` is the mirror: `audienceUids array-contains uid`, minus your own, minus anything more than three hours past its start time.

Joining

`joinPlan()` writes exactly three fields — `attendeeUids`, `attendeeNames.{uid}`, `joinedAt.{uid}` — because that's precisely the shape the security rule accepts (§9).

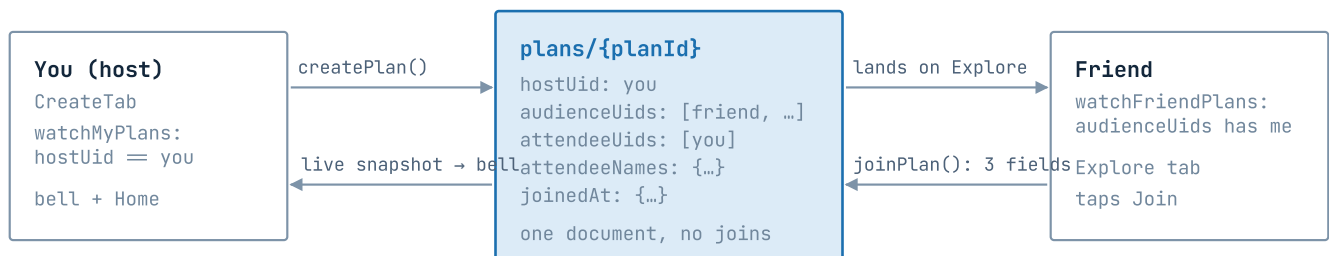
There is **no optimistic local update**. The write goes out, and the live snapshot that comes back is what moves the plan onto Home. One source of truth, no reconciliation code.

THE STALENESS PLAN

A bottom sheet keeps the plan object it was opened with, and that copy goes stale the moment the roster changes — including from the viewer's own tap on Join. So `PlanFeedProvider` exposes `latest(plan)`, which re-looks-up the plan by ID in the live lists. Sheets render `latest(plan)`, never the snapshot they were handed.

Cancelling

`cancelPlan()` sets a `cancelled` flag; it does not delete the document. That's not laziness — it's forced by §7. Because notifications are derived from documents that exist, deleting the plan would delete the "it's cancelled" message along with it. The flag lets the plan vanish from every Home and Explore tab while still carrying word of its own cancellation to the people who were invited.



Nothing is pushed. Both sides hold a standing query on the same document, so one write is seen by everyone whose filter still matches it.

Fig. 3 — One document, four listeners, no messaging layer. The host's "someone joined" notification and the friend's Join button are the same write observed from two sides.

07

NOTIFICATIONS

The feed is derived, not stored

If you only get to explain one thing in an interview, make it this one. It's the most unusual decision in the codebase and it has a crisp justification.

There is no notifications collection. `buildNotifs()` in `notifs.ts` is a pure function that takes the lists the app is *already subscribed to* and returns a sorted feed:

ROW TYPE	DERIVED FROM
<code>request</code>	an incoming <code>friendRequests</code> document
<code>plan</code>	a plan in <code>friendPlans</code>
<code>cancel</code>	a friend's plan with <code>cancelled: true</code>
<code>update</code>	a friend's plan with <code>lastEditKind + updatedAt</code>
<code>suggest</code>	an entry in <code>suggestions</code> on a plan you host
<code>join</code>	a roster entry on your plan that isn't you

The standard approach — a `notifications/{uid}` document written per recipient — needs a server-side fan-out, which means Cloud Functions, which means a billing account. Deriving costs nothing extra, and it **cannot drift out of sync** with the plans it describes: the bell lights up in the same snapshot that puts the plan on Explore, because it's the same data.

Stable IDs are what make it work

Notification IDs are derived from content, never from a counter: `plan:{planId}` , `join:{planId}:{guestUid}` , `edit:{planId}:{updatedAt}` . That's what lets a read receipt survive a reload and a full re-derive.

The `edit:` key includes the timestamp on purpose — a second edit produces a different ID, so it arrives as a new unread row instead of being silently marked read by the receipt left over from the first one.

Read receipts

Receipts live in `localStorage` under `sidequest:notifsRead:{uid}` , capped at the last 300 IDs. They're held in a **module-level store** read

through `useSyncExternalStore` , not in component state — because the bell in the header and the drawer itself both call `useNotifs()` , and with a copy each, the drawer marking everything read would leave the badge still lit.

OWN THE TRADE-OFF

Two real costs, and you should name them before the interviewer does. **Nothing persists** once a plan ages out of the feed — there's no notification history. And **read state is per-browser**, not per-account, so signing in on your phone shows everything unread again. Both are acceptable for a feed that's about what's happening in the next few hours; neither would survive a real product.

THE USUAL WAY — NOT BUILT



WHAT RUNS

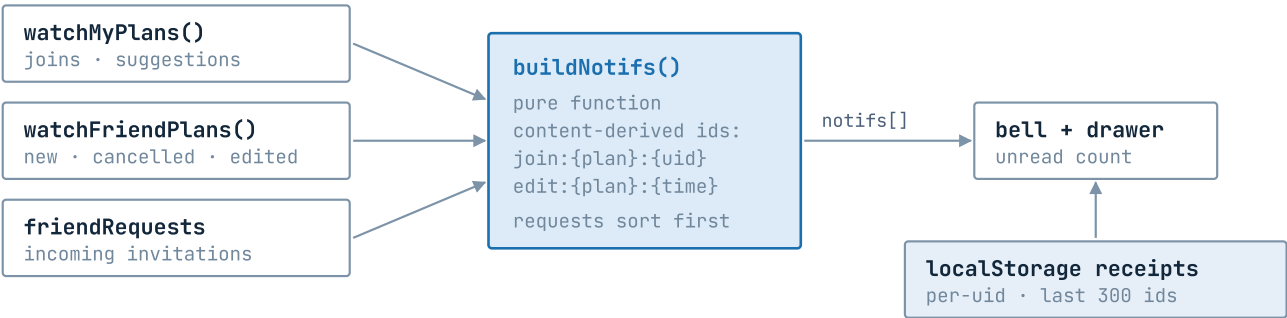


Fig. 4 — The difference is the middle of the top row. Removing the server means removing the write amplification too: a plan shared with twenty friends is one document, not twenty-one. What you give up is history and cross-device read state.

08 Edits and suggestions

FLEX PLANS

A plan can be marked flexible on time, location, or both. That opens a

A plan can be marked flexible on time, location, or both. That opens a negotiation loop between the host and the people invited.

The host moves it

`updatePlan()` changes only time and place, and does one thing beyond the obvious: it stamps `lastEditKind` (`"time"` | `"location"` | `"both"`) alongside `updatedAt` . Since notifications are derived from the documents themselves, an edit without those two fields would be *invisible* — the plan would quietly change underneath everyone and nothing would say so. An edit that changes nothing returns early rather than bumping the document for no reason.

A friend proposes a change

`suggestChange()` writes into `suggestions.{yourUid}` on the plan document itself. Three things fall out of storing it there rather than in its own collection:

- the host is **already subscribed** to their own plans, so the suggestion arrives in a snapshot they're paying for anyway
- no new query means **no composite index** to go create
- one person holds **one open suggestion per plan**, by construction — a second write lands on the same key

A time suggestion carries both a display label and a real `startsAt` , because *"Tomorrow 9:40 AM"* is for reading, not parsing — accepting one has to set an actual timestamp, and you can't recover it from the label.

Accepting

`acceptSuggestion()` is a single write doing three jobs: move the plan, delete the answered suggestion, and stamp `lastEditKind` / `updatedAt` . That last part means accepting a suggestion announces itself to the whole audience through *exactly the same path* a manual edit does — one notification code path, not two.

Related detail: when a new location has no coordinates (someone typed

it rather than picking a place), `lat / lng` are `deleteField()` 'd rather than left behind. Otherwise every card would keep measuring the distance to somewhere the plan no longer is.

The shell enforces the fork before either sheet opens: `openEdit` bails unless you host the plan, `openSuggest` bails if you do host it or it isn't flexible. The rules would reject the write anyway — failing early in the UI is just kinder than failing at the server.

09

AUTHORIZATION

The security rules are the backend

`firestore.rules` is 160 lines and it is doing the job an API layer normally does. Two functions in it are genuinely clever and are the strongest thing to walk an interviewer through line by line.

`isRsvp()` — letting a guest edit a document they don't own

An invited friend needs to add themselves to a plan's roster without being able to touch anything else on it. The rule checks four things:

1. the caller is in `audienceUids`
2. `diff().affectedKeys().hasOnly(['attendeeUids', 'attendeeNames', 'join])` — the write touches those three fields and nothing else
3. the array grew by exactly one and now contains the caller's uid (a join), *or* shrank by exactly one and no longer does (a leave)
4. reads use `.get('attendeeUids', [])` with a default, so a plan written before the roster field existed doesn't error

WHY CHECK THE SIZE

Membership alone isn't enough. Without the size check, a caller could smuggle a *removal of someone else* through alongside adding themselves — add one, remove one, and the set still contains you. Adding one and removing one leaves the size unchanged, which neither branch permits.

`isSuggestion()` — the same trick, one level deeper

Same shape, but it diffs twice: the write must touch only the `suggestions` field, *and within that map* it must touch only the caller's own uid. That nested diff is what stops anyone editing, forging, or deleting a suggestion that isn't theirs, while still letting them write their own.

The rest

- `users/{uid}` — read by any signed-in user (that's what powers handle search), write only by the owner
- `usernames/{handle}` — create only, no update or delete, so a claimed handle isn't transferable
- `friendRequests` — readable and deletable by the two parties; creatable only at the derived ID, only as yourself, only to someone else. No update rule at all, because a request is never edited

BE READY FOR THIS ONE

`plans` has allow read: if `signedIn()` — **any authenticated user can read any plan document directly**. The audience restriction is enforced by the shape of the queries, not by the rules. A sharp interviewer will spot it. The honest answer: it's a real gap, it exists because a per-document read rule would break the `array-contains` query pattern the feed depends on, and the correct fix is to scope reads to `request.auth.uid` in `resource.data.audienceUids` and accept that hosted plans then need a separate rule branch.

`places.ts` wraps `Geography` (100 plan, 5,000 credits/day, no cache).

Autocomplete is request-hungry — every keystroke is potentially a round trip — so the module is built around not spending credits:

- nothing fires below `MIN_QUERY = 3` characters
- callers debounce at 300 ms
- an in-process `Map` caches query → results for the life of the page, so backspacing through a word costs nothing the second time
- a `useRef` request counter discards a slow early response that lands after a newer one — otherwise stale results flash in as you keep typing

Proximity bias, and why it's not optional

Unbiased, searching "blue bottle" ranks a reservoir in Wyoming above the café down the street. So results are biased by coordinates resolved once per page load, in a two-tier fallback:

1. **browser geolocation** — but only if permission is *already granted*. It deliberately never triggers the prompt; a permission dialog thrown over a form mid-keystroke is hostile. Onboarding asks at a sane moment instead.
2. **IP lookup** — city-level, costs one credit, needs no prompt.

The nicest detail in the file

A plan pinned to "Current Location" stores your coordinates, which is all a *distance* needs — but a friend reading the card needs a *name*. So `describeCoords()` reverse-geocodes it **once, by the host, at the moment they share**, and that resolved label is what lands in Firestore. One credit per plan, instead of one per reader per render — and the string is then identical everywhere it appears: card, detail sheet, and the "now meets at..." notification.

Better still, the granularity follows the fix behind it. A browser fix names the building; an IP fix only ever names the neighbourhood, because a street address derived from a city-level fix would be a fiction. And it returns `null` rather than throwing — a plan whose label didn't resolve is

still a perfectly good plan, and blocking the share on a geocode would be a poor trade.

Distance itself is plain Haversine in `distanceMiles()`, computed in `PlanFeedProvider` rather than in the query layer — it needs the viewer's position, which can change without the plans changing. It's formatted coarsely on purpose (`"0.3 mi"`, not `"0.28 mi"`) since the underlying fix may be city-level.

11

SHIPPING

Build, PWA, deploy

`npm run deploy` is `vite build && firebase deploy --only hosting`. Vite bundles to `dist/`, Firebase Hosting serves it.

Installable

`vite-plugin-pwa` generates a manifest and a Workbox service worker with `registerType: 'autoUpdate'` — a service worker's whole job is serving cached files, so without auto-update a new deploy would sit unseen behind the last one installed. iOS ignores the web manifest for several things, which is why `index.html` repeats the app title, theme colour, and touch icon as Apple-specific meta tags.

THE SUBTLE BIT

`navigateFallbackDenylist: [/^\/__/]`. It's a single-page app, so any unknown path should fall back to `index.html` — *except* Firebase's own reserved paths. `/__/auth/*` is where Hosting serves the OAuth handler, and letting the service worker answer those with `index.html` silently breaks Google and Apple sign-in. That's the kind of bug that costs an afternoon.

Cache headers

`firebase.json` splits caching by what's fingerprinted: `/assets/**` gets

`max-age=31536000, immutable` because Vite hashes those filenames, while `index.html`, `sw.js`, `registerSW.js` and the manifest get no-cache so a deploy is actually picked up.

Secrets

The Firebase config is committed in `firebase.ts`, and that's correct — web API keys aren't secrets, they identify the project. Authorization comes entirely from the security rules. The Geoapify key is in a gitignored `.env` as `VITE_GEOAPIFY_KEY`, but Vite inlines `VITE_`-prefixed vars into the bundle, so it ships to the browser regardless — which is why it must be domain-restricted in the Geoapify dashboard. Be able to say that; "the key is in `.env` so it's safe" is the wrong answer and interviewers do ask.

12

GAPS

What's honest to say is unfinished

Knowing your own gaps reads as senior. Volunteering them before you're caught reads as confident. These are real:

GAP	WHERE
availability not wired	<code>currentUser.tsx</code> exposes <code>availability</code> , <code>setAvailability</code> and <code>dnd</code> , and <code>users.ts</code> persists status — but <code>ProfileTab</code> holds its status in a local <code>useState</code> . Nothing in the app reads the context value. The plumbing is built; the last wire isn't connected.
hardcoded profile stats	"12 Friends / 28 Plans / 94% Join Rate" are literals in <code>ProfileTab</code> .
Explore search is decorative	It's a styled <code><div></code> with placeholder text, not an input.
"Ping!" does nothing	Button in <code>FriendsTab</code> with no handler.
coincidence feature shelved	"A friend is nearby and free" — commented out consistently across the

	<code>types, friends.ts , the onboarding radius step, and a parallel HomeTab.withCoincidenceAlerts.tsx . It's the reason the radius slider exists.</code>
<code>two orphaned variant files</code>	<code>ProfileTab.withRadius.tsx and HomeTab.withCoincidenceAlerts.tsx aren't imported anywhere.</code>
<code>plans are world-readable</code>	See §9. The audience filter is a query convention, not a rule.
<code>no tests, no CI</code>	Nothing in the repo. <code>buildNotifs()</code> is a pure function of four arrays – it's the obvious first unit test, and worth saying so.
<code>seeded demo data still present</code>	<code>MY_PLANS_TODAY / MY_PLANS_LATER</code> at the bottom of <code>plans.ts</code> , unused.

13

THE
INTERVIEW

Questions you'll get, and how to answer them

"Walk me through the architecture."

Serverless React client on Firebase. Three layers: a `data/` module per domain that owns all Firestore access and exposes a context or hook, screens that only consume those, and one shared `types` file as the contract. Authorization lives entirely in security rules because there's no server to put it in.

"What was the hardest problem?"

Making friendship mutual without a backend. Go to §5 and Fig. 5 — the pending request document acts as a capability that authorizes exactly one write into someone else's private subcollection, for exactly as long as the invitation is open.

"How do notifications work?"

They're derived, not stored — §7. Lead with the constraint (Cloud Functions need billing), then the payoff (no write amplification, can't

drift out of sync), then volunteer the cost (no history, read state is per-browser).

"How would you scale this?"

Three honest answers. Notifications would have to become real documents written by a Cloud Function once fan-out gets large.

`audienceUids` is an array on a document, so it caps out around a few hundred friends. And the Home tab's two-listener merge would want to become one query plus a proper composite index.

"Where's the security risk?"

Don't wait to be asked. Plans are readable by any authenticated user (§9), and the Geoapify key ships in the bundle so it depends on domain restriction (§11). Naming both, unprompted, is worth more than either being absent.

"Why no Redux / React Query / router?"

Firestore's `onSnapshot` already is the cache and the subscription — a client cache layer on top would be a second source of truth to keep in sync. Context is enough because there are exactly three shared concerns. And a five-tab app with no deep links doesn't need URL routing; two pieces of state in the shell cover it. Be ready to add: deep links into a plan would be the reason to add a router.

"What would you do differently?"

Tighten the plan read rule first. Then write tests around `buildNotifs()` and the rules themselves — the Firebase emulator can test rules directly, and rules are exactly the kind of logic that's dangerous untested. Then move read receipts onto the user document so they follow the account.

ONE-LINE VERSION

"It's a serverless React app on Firestore where the security rules do the work an API layer normally does — including two rules that let an invited guest write precisely one field on a plan they don't own —

and where the notification feed is computed on the client from the queries the app is already running, instead of being fanned out by a server."

Read against commit 711699e6 · ~7,200 lines across src/app · sidequest-b73eb